

Assuntos a serem tratados:

- Modelagem e síntese de sistemas digitais a partir de um dado *testbench*.
- Comparações entre números, sem e com sinal, utilizando *VHSIC Hardware Description Language* (VHDL).
- Instrução de atribuição de sinais em VHDL usando atribuição condicionada (*when else*).
- Desenvolvimento de componentes genéricos usando VHDL.

Objetivos:

- Construir um comparador de magnitude usando comandos concorrentes, como a estrutura *when else*.
- Utilizar a linguagem VHDL para comparar resultados envolvendo representações numéricas sem e com sinal, estes últimos representados em complemento de dois.
- Entender o conceito de construção de componentes genéricos em VHDL.

Atividade prática em sala de aula:

- **Construção do módulo *Comparator*.**

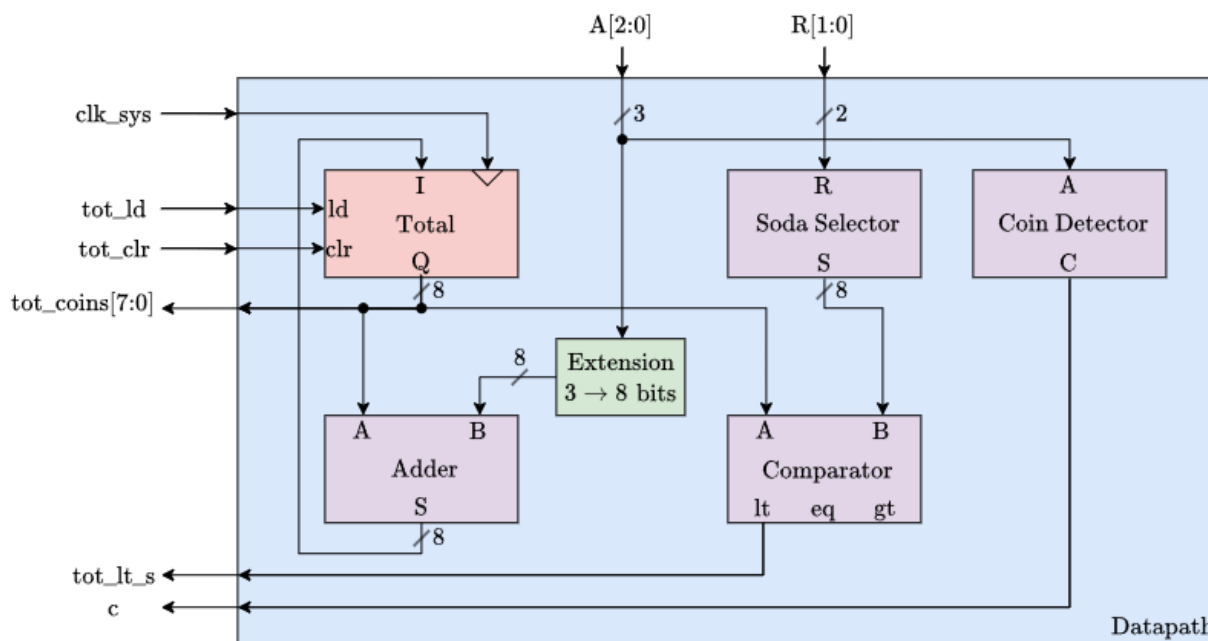


Figura 1 – Caminho de dados que contém o componente *Comparator*.

- 1) Leia o arquivo de *testbench* (***tb_comparator.vhd***) fornecido e entenda como está sendo feito o teste do comparador de magnitude e igualdade a ser projetado.
- 2) Crie um projeto no Quartus denominado ***comparator*** em uma pasta previamente escolhida, selecionando a família e o dispositivo FPGA presentes na placa didática de desenvolvimento DE10-Lite.
- 3) Crie o arquivo ***comparator.vhd*** contendo uma entidade com as mesmas especificações do componente ***comparator*** declarado no arquivo ***tb_comparator.vhd***.
- 4) Implemente, no arquivo ***comparator.vhd***, uma arquitetura em VHDL para um comparador de palavras *A* e *B* de 8 bits, contendo três saídas — ***greater than (gt)***, ***equal to (eq)*** e ***less than (lt)*** — responsáveis por indicar, respectivamente, as condições $A > B$, $A = B$ e $A < B$. Para isso, utilize a estrutura concorrente ***when else*** e considere que os números *A* e *B* estão representados em complemento de dois.
- 5) Compile a descrição do sistema para o dispositivo FPGA definido no início do projeto.
- 6) Acesse a opção ***RTL Viewer*** no menu ***Tools*** → ***Netlist Viewers*** e observe se o diagrama mostrado é coerente com o esperado.

- 7) Examine o código do *testbench* e determine quantos *bits* possui o componente instanciado na simulação. Esse componente possui uma largura de *bits* menor que o comparador criado no arquivo **comparator.vhd**? Se sim, qual é a razão para tal escolha?
- 8) Realize a simulação funcional no ModelSim utilizando o arquivo de *testbench* (**tb_comparator.vhd**) em conjunto com o arquivo **comparator.vhd** desenvolvido. Em seguida, analise as formas de ondas obtidas e verifique se o comportamento do comparador corresponde ao esperado:

	Msgs				
/tb_comparator/pin_a	0000	0000	1000	0111	
/tb_comparator/pin_b	0000	0000	0111	1000	0001
/tb_comparator/pin_gt	0				
/tb_comparator/pin_eq	1				
/tb_comparator/pin_lt	0				

- 9) Modifique a arquitetura desenvolvida para que as entradas A e B passem a ser interpretadas como valores sem sinal, em vez de números representados em complemento de dois. Em seguida, realize a simulação funcional do projeto para o mesmo *testbench* **tb_comparator.vhd** e verifique se o comportamento da arquitetura modificada apresenta o seguinte comportamento:

	Msgs				
/tb_comparator/pin_a	0000	0000	1000	0111	
/tb_comparator/pin_b	0000	0000	0111	1000	0001
/tb_comparator/pin_gt	0				
/tb_comparator/pin_eq	1				
/tb_comparator/pin_lt	0				

- 10) Descreva como foram implementadas as diferentes representações numéricas — complemento de dois e representação sem sinal — de modo que as duas simulações apresentem resultados distintos utilizando o mesmo *testbench* e as mesmas entradas.
- 11) Crie um novo projeto denominado **comparator_4** (comparador de 4 *bits*), utilizando o componente comparador desenvolvido anteriormente sem alterar seu código. Para isso, utilize a construção **generic map** para instanciar o comparador de 4 *bits* e criar o arquivo **comparator_4.vhd**. Em seguida, adicione o arquivo **comparator.vhd** ao novo projeto, sem movê-lo de seu diretório de origem, por meio do menu **Project → Add/Remove Files in Project**. Por fim, compile o projeto e teste o componente na placa de desenvolvimento DE10-Lite.

Rubrica de avaliação:

- Validação, via simulação funcional, do comparador de 8 *bits* com entradas em complemento de dois: _____
- Validação, via simulação funcional, do comparador de 8 *bits* com entradas sem sinal: _____
- Validação, via implementação na placa DE10-Lite, do comparador de 4 *bits*: _____